

End-to-End Python Notes for Data Analysis

💡 The complete Python data analysis pipeline, start to finish — setup, load, inspect, clean, transform, aggregate, merge, visualize, and export. Everything in the order it actually happens in real work, with syntax and the traps at each stage.

1. Setup and Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', None) # see every column, not truncated
pd.set_option('display.width', None)
```

🎯 The two `set_option` lines aren't optional in practice — without them, wide DataFrames get truncated in the output and you silently miss columns sitting off-screen.

2. Loading Data

Source	Command
CSV	<code>pd.read_csv('file.csv')</code>
Excel	<code>pd.read_excel('file.xlsx', sheet_name='Sheet1')</code>
SQL	<code>pd.read_sql("SELECT * FROM orders", conn)</code>
JSON	<code>pd.read_json('file.json')</code>
Clipboard	<code>pd.read_clipboard()</code> — genuinely useful for quick pastes

```
df = pd.read_csv('data.csv', parse_dates=['order_date'])
```

⚠ **parse_dates at load time saves a step later.** Without it, date columns load as plain text — you'd have to convert them with `pd.to_datetime()` afterward anyway, so do it up front.

3. First Look at the Data

```
df.shape
df.head() / df.tail()
df.info()
df.describe(include='all')
df.dtypes
df.sample(5)
```

🎯 Run all of these before writing any real analysis. `.info()` catches wrong dtypes and null counts in one shot — a numeric column silently loaded as text breaks every calculation downstream, and this is where you catch it in seconds.

⚠ **describe(include='all'), not plain describe().** Plain `describe()` silently skips every text/categorical column entirely.

4. Data Cleaning

Missing values:

```
df.isnull().sum()
df['col'] = df['col'].fillna(df['col'].mean()) # only if
0/mean genuinely makes sense
df = df.dropna(subset=['critical_column'])
```

⚠ Never `fillna(0)` blindly — decide per column what a NULL actually means first. Missing "sales" might mean zero. Missing "age" never does.

Duplicates:

```
df.duplicated().sum()
df = df.drop_duplicates()
df = df.drop_duplicates(subset=['customer_id', 'order_date']) # specific columns
```

Data types:

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['amount'] = df['amount'].astype(float)
df['category'] = df['category'].astype('category') # save
s memory on repeated text
```

Text cleaning:

```
df['name'] = df['name'].str.strip().str.lower()
```

⚠ Untrimmed, inconsistently-cased text silently splits one real category into several — "Sales", " sales", "Sales " all count separately until normalized.

5. Data Transformation and Feature Engineering

```
df['profit_margin'] = df['profit'] / df['revenue']
df['tier'] = df['amount'].apply(lambda x: 'high' if x > 500
else 'low')
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.month_name()
df['is_weekend'] = df['order_date'].dt.dayofweek >= 5

# Binning a continuous variable
df['age_group'] = pd.cut(df['age'], bins=[0, 18, 35, 60, 100],
labels=['<18', '18-35', '36-60', '60+'])
```

🕒 .dt accessor unlocks every date/time property once a column is a real datetime — year, month, day name, weekday number, all without writing conversion logic yourself.

🎯 `pd.cut()` is the fast way to bucket a continuous variable into readable categories — genuinely faster than a chain of nested IF-style conditions.

6. GroupBy and Aggregation

```
df.groupby('region')['amount'].sum()
df.groupby('region')['amount'].agg(['sum', 'mean', 'count'])
df.groupby(['region', 'category'])['amount'].sum()  # multi-level grouping

df.groupby('region').agg(
    total_sales=('amount', 'sum'),
    avg_order=('amount', 'mean'),
    order_count=('order_id', 'count')
)
```

🎯 The named-aggregation syntax (`total_sales=('amount','sum')`) is worth learning early — it produces clean, readable column names instead of the default multi-index mess a plain `.agg(['sum','mean'])` leaves behind.

Pivot table equivalent:

```
df.pivot_table(values='amount', index='region', columns='month', aggfunc='sum')
```

7. Merging and Combining Data

```
df.merge(customers, on='customer_id', how='left')
pd.concat([df_jan, df_feb, df_mar])  # stacking, not joining
```

how=	Behavior
inner	only matched rows on both sides

how=	Behavior
left	every row from the left table, NULL-filled on unmatched right
right	every row from the right table
outer	everything from both sides

⚠ **how= maps directly to SQL join types.** Get it wrong and rows silently vanish — exactly like picking the wrong SQL join.

8. Visualization

```
df['amount'].hist(bins=30)
sns.boxplot(x='category', y='amount', data=df)
sns.scatterplot(x='col1', y='col2', hue='category', data=df)
df.groupby('region')['amount'].sum().plot(kind='bar')
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
```

🎯 Four chart types cover most real exploratory work: histogram (distribution), boxplot (spread/outliers), scatter (relationships), bar (category comparison). Correlation heatmap is the fifth — always run it before a full pairplot.

9. Basic Statistics in Code

```
df['amount'].mean(), df['amount'].median(), df['amount'].std()
df['amount'].skew() # >1 or <-1 = don't trust the mean
df.corr(numeric_only=True)

# Outlier detection, IQR method
Q1, Q3 = df['amount'].quantile(0.25), df['amount'].quantile(0.75)
IQR = Q3 - Q1
```

```
outliers = df[(df['amount'] < Q1-1.5*IQR) | (df['amount'] > Q3+1.5*IQR)]
```

🎯 Check `.skew()` before reporting any average. A skew beyond ± 1 means the mean is being pulled by outliers — median is the more honest number to report.

10. Exporting Results

```
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', sheet_name='Results', index=False)

# Multiple sheets in one Excel file
with pd.ExcelWriter('report.xlsx') as writer:
    summary.to_excel(writer, sheet_name='Summary')
    details.to_excel(writer, sheet_name='Details')
```

⚠️ **index=False is the flag everyone forgets at least once.** Without it, pandas writes an extra unnamed column containing the DataFrame's internal row index into your export.

🏁 The Full Pipeline at a Glance

Stage	Core Commands	The One Trap to Remember
🟦 Setup	<code>pd.set_option()</code>	wide DataFrames truncate silently without it
🟢 Loading	<code>read_csv</code> , <code>read_sql</code> , <code>parse_dates</code>	parse dates at load time, not after
🔴 First Look	<code>.info()</code> , <code>.describe(include='all')</code>	plain <code>describe()</code> skips text columns
🧹 Cleaning	<code>fillna</code> , <code>dropna</code> , <code>drop_duplicates</code> , TRIM-equivalent	<code>fillna(0)</code> is a lie unless zero is genuinely correct
🟠 Transformation	<code>.apply()</code> , <code>.dt</code> accessor, <code>pd.cut()</code>	<code>.dt</code> only works once a column is real datetime

Stage	Core Commands	The One Trap to Remember
 GroupBy	<code>.groupby().agg()</code>	use named aggregation for clean output columns
 Merging	<code>.merge(how=)</code> , <code>pd.concat()</code>	<code>how=</code> silently drops rows if picked wrong
 Visualization	<code>hist</code> , <code>boxplot</code> , <code>scatter</code> , <code>bar</code> , <code>heatmap</code>	correlation only catches LINEAR relationships
 Statistics	<code>.skew()</code> , <code>.corr()</code> , IQR outliers	check skew before trusting any mean
 Export	<code>to_csv(index=False)</code>	forgetting <code>index=False</code> adds a stray column

You're ready when you can run this entire ten-stage pipeline on a dataset you've never seen — load it, clean it, transform it, group it, merge in a second table, visualize it, and export a clean result — in one sitting, without pausing to look up syntax. That's what "end to end" actually means here: not ten isolated skills, but one continuous, repeatable workflow.